



Reporte Ciudadano

Municipalidad de Gualeguaychú — Pedido de aplicación móvil

Documento redactado por Claudia Benítez, coordinadora del Centro de Atención al Vecino de la Municipalidad de Gualeguaychú, Entre Ríos, para el equipo de desarrollo.

Cómo llegamos hasta acá

Coordino el Centro de Atención al Vecino desde 2019. Somos seis personas atendiendo un 0800, un mostrador y, desde la pandemia, un WhatsApp que nunca deja de sonar. Por acá pasa todo lo que se rompe en la ciudad: baches, luminarias quemadas, ramas caídas, basura acumulada, semáforos que no andan, veredas rotas.

El sistema que tenemos es una planilla compartida donde cargamos a mano lo que nos dicen por teléfono. El vecino corta y no se lleva nada: ni un número, ni una constancia, ni una fecha. Después llama a los quince días para preguntar y nosotras no sabemos qué contestarle, porque la planilla dice "bache en Urquiza al 1200" y en Urquiza al 1200 hay ocho cuadras.

El trimestre pasado me terminó de decidir. Pasaron estas tres cosas:

- Cargamos catorce veces el mismo bache de Rocamora y Bolívar, porque cada vecino que llamaba era un reclamo nuevo. Obras mandó la cuadrilla dos veces al mismo lugar.
- Una señora reclamó por una rama que estaba por caerse arriba de un cable. Se anotó mal la calle. La rama cayó tres semanas después y cortó la luz de media manzana.
- El intendente se enteró por Facebook de un pozo que nosotras teníamos cargado hacía dos meses. Nadie le había avisado, porque nuestra planilla no la mira nadie más que nosotras.

Lo que necesito es que el vecino pueda reportar desde el celular, en el momento y parado frente al problema, con una foto y la ubicación exacta. Que se lleve un número de seguimiento y que le avisemos cuando pasa algo. Y que del otro lado, el operador municipal tenga una lista ordenada en vez de una planilla de mil filas.

Qué necesito que la app haga

Lo pienso en seis áreas. Para cada una anoté qué necesito poder hacer y qué hay que guardar.

1. Crear un reporte

Es lo primero y lo más importante. Tiene que poder hacerse en menos de un minuto, parado en la vereda.

- Elegir el tipo de problema de una lista corta: bache, luminaria, basura, rama o árbol, agua o cloaca, semáforo, vereda, otro.
- **Sacar una foto es obligatorio.** Sin foto no hay reporte. Es lo único que nos evita las discusiones.
- Poder agregar una segunda foto si con una no se entiende.
- Que la ubicación se tome sola del GPS, y que el vecino pueda corregir el punto arrastrándolo en el mapa si el GPS le erró.

- Describir el problema escribiendo, o **grabando una nota de voz**. Mucha gente grande no escribe en el celular pero habla sin problema.
- Que al terminar le quede un número de seguimiento visible.

Esto va a pasar seguro: el vecino saca la foto en un lugar sin señal —tenemos barrios enteros así— y después se va. El reporte no se puede perder. Tiene que quedar guardado en el teléfono y subirse solo cuando vuelva la conexión, sin que el vecino tenga que acordarse de nada.

2. Seguir el reclamo

- Ver la lista de reportes propios, del más nuevo al más viejo.
- Entrar a uno y ver en qué estado está y qué le fue pasando.
- Los estados son cinco: *recibido*, *en revisión*, *asignado a cuadrilla*, *resuelto*, *rechazado*.
- Cuando lo resolvemos, quiero poder subir una foto del arreglo. Es lo que más nos van a agradecer.
- Un reclamo rechazado tiene que decir por qué. "Rechazado" a secas genera más enojo que no contestar.
- Poder mostrar el reporte en el mostrador con un código QR, para que la chica de la ventanilla lo abra sin tipear nada.

3. El mapa de la ciudad

- Ver todos los reportes públicos sobre un mapa, con un color por estado.
- Filtrar por tipo de problema y por estado.
- Ver qué hay reportado cerca de donde está la persona.
- **Antes de crear un reporte nuevo, mostrarle los que ya existen a menos de 50 metros** y preguntarle si es el mismo. Ahí se me van los catorce baches repetidos.
- Poder sumarse a un reporte existente en vez de crear otro. Cuantos más vecinos se suman, más arriba va en la lista de Obras.

4. La bandeja del operador

Esta parte la usamos nosotras, no el vecino. La misma app, pero quien entra con cuenta de operador ve otra cosa.

- Lista de reportes con filtros por estado, tipo y zona.
- Cambiar el estado de un reporte y dejar escrito por qué.
- Asignarlo a una cuadrilla.
- Marcar un reporte como duplicado de otro.
- Subir la foto del arreglo al cerrarlo.

No hace falta que sea una app distinta. Alcanza con que la app sepa quién entró y le muestre lo que le corresponde. Al vecino no se le puede mostrar el botón de cambiar estados.

5. Los avisos

- Avisarle al vecino cada vez que cambia el estado de un reporte suyo.
- Avisarle cuando alguien se suma a un reporte que él creó.
- Que pueda apagar los avisos si no los quiere.

📍 6. Zonas y cuadrillas

La ciudad está dividida en cuatro zonas de mantenimiento, y cada una tiene sus cuadrillas. Necesito:

- Que el reporte quede asociado automáticamente a la zona según dónde cayó el punto.
- Ver el listado de cuadrillas y a qué zona pertenecen.
- Filtrar la bandeja por zona.

Cómo me imagino que se usa

Crear un reporte

```
+-----+
| <  Nuevo reporte          |
+-----+
| Que pasa?                 |
| [Bache] [Luz] [Basura] [Rama] |
| [Agua] [Semaforo] [Vereda] [Otro] |
+-----+
| Foto (obligatoria)       |
| [  tomar foto  ] [ galeria ] |
+-----+
| Donde                     |
| Rocamora 1240 - a 12 m de vos |
| [ corregir en el mapa ]    |
+-----+
| Contanos                  |
| [ escribir ] o [ grabar audio ] |
+-----+
|           [  E N V I A R  ]           |
+-----+
```

El detalle de un reclamo

```
+-----+
| <  Reporte GCHU-2026-00412  [QR] |
+-----+
| Bache - Rocamora 1240       |
| Estado: ASIGNADO A CUADRILLA |
| 3 vecinos se sumaron       |
+-----+
|           [ fotografia ]      |
+-----+
| Que fue pasando            |
| 14/09  Recibido             |
| 15/09  En revision          |
|         "Verificado en el lugar" |
| 18/09  Asignado - Cuadrilla 2 |
+-----+
```

Cosas importantes para cómo lo vamos a usar

- **Sin señal tiene que andar igual.** Los barrios del norte y toda la zona de las quintas tienen señal intermitente. El reporte se guarda y se sube solo.
- **Un minuto, no más.** Si crear un reporte lleva cinco pasos y tres pantallas, la gente vuelve al WhatsApp.
- **La foto es obligatoria y no se negocia.** Si el vecino no da permiso de cámara, no puede reportar, y la app tiene que explicárselo con buenos modos y ofrecerle abrir la configuración.

- **Si niega la ubicación**, que igual pueda reportar poniendo el punto a mano en el mapa.
- **Gente grande**. Buena parte de quienes reclaman tienen más de 60. Letra grande, botones grandes, y la nota de voz como alternativa a escribir.
- **Nada de datos sensibles a la vista**. En el mapa público se ve el reporte, no quién lo hizo.
- **Android y iPhone**, con mayoría de Android.

Lo que no entra en esta primera versión

- Turnos, trámites y pagos municipales. Esto es solo reclamos.
- Chat entre el vecino y el operador. Los cambios de estado con comentario alcanzan.
- Estadísticas y tableros para las secretarías. Lo vamos a querer, pero después.
- Integración con el sistema de expedientes de la Municipalidad.
- Encuesta de satisfacción al cerrar el reclamo.

Cómo quiero que trabajemos

- **No sé de sistemas**. Si algo de lo que pedí no se puede, es carísimo o no se entiende, díganmelo de frente.
- **Prefiero que pregunten mucho ahora** y no que me entreguen en noviembre algo que no sirve.
- **Este documento tiene huecos**. Lo escribí yo, entre llamados. Van a encontrar cosas que faltan o que se contradicen. Cuando las encuentren, avisen: no las resuelvan por su cuenta y me lo cuenten después.
- **Voy a pedir cambios**. Cuando lo haga, quiero que me digan qué se cae o qué se atrasa. "Sí, no hay problema" a todo no me sirve.
- **Muéstrenme algo temprano**, aunque sea feo y con datos inventados.

Algunas cuestiones técnicas que me pidieron incluir

El equipo docente me pidió que deje escritas algunas definiciones técnicas, para que no haya malentendidos.

Con qué se construye

Qué	Con qué
Framework	React Native con Expo
Lenguaje	TypeScript
Navegación	Expo Router
Estilos	StyleSheet . Unistyles o NativeWind son opcionales
Entrega	Build de producción con EAS

Capacidades del dispositivo que tiene que usar

Capacidad	Paquete	Para qué en esta app
Cámara	<code>expo-camera</code> (<code>CameraView</code>)	Foto obligatoria del problema y lectura del QR en el mostrador
Galería	<code>expo-image-picker</code>	Adjuntar una foto sacada antes
Ubicación y mapa	<code>expo-location</code> + <code>react-native-maps</code>	Punto exacto del reclamo, corrección a mano y mapa de la ciudad
Notificaciones	<code>expo-notifications</code>	Aviso ante cada cambio de estado
Persistencia y red	<code>expo-sqlite</code> , <code>expo-sqlite/kv-store</code> , <code>expo-network</code>	Cola de reportes sin subir, borrador en curso, sesión y preferencias
Seguridad	<code>expo-secure-store</code> + <code>expo-local-authentication</code>	Token de sesión guardado; el operador reingresa con huella
Sensores o háptica	<code>expo-haptics</code> , <code>expo-sensors</code>	Vibración al confirmar el envío y al detectar un reporte duplicado cerca
Sistema de archivos	<code>expo-file-system</code>	Guardar la foto y el audio hasta que se suben
Multimedia	<code>expo-audio</code>	Grabar y reproducir la descripción hablada del problema

Tres cosas que la documentación vieja de internet dice mal. `expo-av` no existe más: el audio es `expo-audio` y el video, `expo-video`. La API vieja de `expo-file-system` tira error en tiempo de ejecución: se usa `File`, `Directory` y `Paths`. Los íconos son `@react-native-vector-icons/ionicons`, no `@expo/vector-icons`.

Cómo va a venir la información

La API la provee la cátedra y va a estar disponible más adelante en el cuatrimestre. Hasta entonces, la app se construye contra datos inventados. Reglas para que el cambio después sea barato:

- Los tipos de este documento se escriben una sola vez en `src/tipos/` y no se tocan cuando llegue la API.
- Los datos falsos van en `src/mocks/`, tipados con esos mismos tipos. Si el mock no compila contra el tipo, está mal el mock.
- Ninguna pantalla importa el mock directamente. Toda lectura y escritura pasa por una capa de servicios (`src/servicios/reportes.ts`) que hoy devuelve el mock y mañana hace `fetch`.
- Los servicios son asíncronos desde el primer día, aunque el mock responda al instante.
- Los mocks tienen que incluir los casos feos: un reporte sin audio, uno rechazado con motivo largo, una lista vacía, un reporte con cinco cambios de estado, y un error de red.

La respuesta de la API va a tener siempre esta forma:

```
// éxito
{ "datos": [ ... ], "meta": { "total": 128, "pagina": 1, "porPagina": 20 } }

// error
{ "error": { "codigo": "FOTO_REQUERIDA", "mensaje": "El reporte necesita al menos una foto." } }
```

Convenciones de los datos, sin excepción:

- Los identificadores son **cadenas de texto**, no números.
- Las fechas y horas viajan como texto en **ISO 8601 con zona** (`"2026-09-14T10:22:00-03:00"`). Nunca un objeto `Date` ni un número.
- Las claves van en `camelCase`.
- Un campo que puede no tener valor viene como `null`, no ausente y no como cadena vacía.
- La sesión viaja en la cabecera `Authorization: Bearer <token>`. El token se guarda en `expo-secure-store`, nunca en `kv-store`.

Resumen — qué entidades maneja la app

Referencia rápida de todo lo que la app guarda y muestra. Para cada entidad va la tabla de campos, el tipo de TypeScript y un ejemplo listo para pegar en el archivo de mocks.

Reporte

Propiedad	Tipo	Ejemplo
id	string	"rep-00412"
codigo	string	"GCHU-2026-00412"
tipoId	string	"tip-bache"
descripcion	string null	"Pozo grande, pasa el agua"
audioUrl	string null	nota de voz, si la grabó
fotos	Foto[]	al menos una
coordenadas	Coordenadas	{ latitud: -33.00, longitud: -58.51 }
direccion	string	"Rocamora 1240"
zonaId	string	"zon-norte"
estado	EstadoReporte	"asignado"
autorId	string	"usr-084"
cuadrillaId	string null	"cua-02"
duplicadoDe	string null	null
adhesiones	number	3
creadoEn	string (ISO)	"2026-09-14T10:22:00-03:00"
sincronizado	boolean	false mientras está en la cola

```

export interface Coordenadas {
  latitud: number;
  longitud: number;
}

export interface Foto {
  id: string;
  url: string;          // o ruta local mientras no se subió
  momento: "problema" | "arreglo";
}

export type EstadoReporte =
  | "recibido" | "en_revision" | "asignado" | "resuelto" | "rechazado";

export interface Reporte {
  id: string;
  codigo: string;
  tipoId: string;
  descripcion: string | null;
  audioUrl: string | null;
  fotos: Foto[];
  coordenadas: Coordenadas;
  direccion: string;
  zonaId: string;
  estado: EstadoReporte;
  autorId: string;
  cuadrillaId: string | null;
  duplicadoDe: string | null;
  adhesiones: number;
  creadoEn: string;
  sincronizado: boolean;
}

```

```

{
  "id": "rep-00412",
  "codigo": "GCHU-2026-00412",
  "tipoId": "tip-bache",
  "descripcion": "Pozo grande en la mano hacia el centro, pasa el agua.",
  "audioUrl": null,
  "fotos": [
    { "id": "fot-901", "url": "https://api.gchu.gob.ar/f/901.jpg", "momento": "problema" }
  ],
  "coordenadas": { "latitud": -33.0089, "longitud": -58.5142 },
  "direccion": "Rocamora 1240",
  "zonaId": "zon-norte",
  "estado": "asignado",
  "autorId": "usr-084",
  "cuadrillaId": "cua-02",
  "duplicadoDe": null,
  "adhesiones": 3,
  "creadoEn": "2026-09-14T10:22:00-03:00",
  "sincronizado": true
}

```

TipoDeReporte

Propiedad	Tipo	Ejemplo
id	string	"tip-bache"
nombre	string	"Bache"
icono	string	"warning-outline"
color	string	"#C1440E"
areaResponsable	string	"Obras Públicas"

```
export interface TipoDeReporte {  
  id: string;  
  nombre: string;  
  icono: string;  
  color: string;  
  areaResponsable: string;  
}
```

```
{ "id": "tip-bache", "nombre": "Bache", "icono": "warning-outline",  
  "color": "#C1440E", "areaResponsable": "Obras Públicas" }
```

CambioDeEstado

Propiedad	Tipo	Ejemplo
id	string	"cam-1177"
reporteId	string	"rep-00412"
estado	EstadoReporte	"en_revision"
comentario	string null	"Verificado en el lugar"
operadorId	string null	"usr-003"
fechaHora	string (ISO)	"2026-09-15T08:40:00-03:00"

```
export interface CambioDeEstado {  
  id: string;  
  reporteId: string;  
  estado: EstadoReporte;  
  comentario: string | null;  
  operadorId: string | null;  
  fechaHora: string;  
}
```

```
{ "id": "cam-1177", "reporteId": "rep-00412", "estado": "en_revision",  
  "comentario": "Verificado en el lugar", "operadorId": "usr-003",  
  "fechaHora": "2026-09-15T08:40:00-03:00" }
```

Usuario

Propiedad	Tipo	Ejemplo
id	string	"usr-084"
nombre	string	"Norma Pereyra"
email	string	"norma@mail.com"
telefono	string null	"3446-412233"
rol	Rol	"vecino"
zonaId	string null	sólo para operadores
avisosActivos	boolean	true
creadoEn	string (ISO)	"2026-08-30T19:10:00-03:00"

```
export type Rol = "vecino" | "operador";
```

```
export interface Usuario {  
  id: string;  
  nombre: string;  
  email: string;  
  telefono: string | null;  
  rol: Rol;  
  zonaId: string | null;  
  avisosActivos: boolean;  
  creadoEn: string;  
}
```

```
{  
  "id": "usr-084", "nombre": "Norma Pereyra", "email": "norma@mail.com",  
  "telefono": "3446-412233", "rol": "vecino", "zonaId": null,  
  "avisosActivos": true, "creadoEn": "2026-08-30T19:10:00-03:00"  
}
```

Zona

Propiedad	Tipo	Ejemplo
id	string	"zon-norte"
nombre	string	"Zona Norte"
limite	Coordenadas[]	polígono que la delimita
referente	string	"Corralón Norte"

```
export interface Zona {  
  id: string;  
  nombre: string;  
  limite: Coordenadas[];  
  referente: string;  
}
```

```
{  
  "id": "zon-norte", "nombre": "Zona Norte", "referente": "Corralón Norte",  
  "limite": [  
    { "latitud": -32.99, "longitud": -58.53 },  
    { "latitud": -32.99, "longitud": -58.49 },  
    { "latitud": -33.02, "longitud": -58.49 } ]  
}
```

Cuadrilla

Propiedad	Tipo	Ejemplo
id	string	"cua-02"
nombre	string	"Cuadrilla 2 – Bacheo"
zonaId	string	"zon-norte"
especialidad	string	"pavimento"
activa	boolean	true

```
export interface Cuadrilla {  
  id: string;  
  nombre: string;  
  zonaId: string;  
  especialidad: string;  
  activa: boolean;  
}
```

```
{ "id": "cua-02", "nombre": "Cuadrilla 2 – Bacheo", "zonaId": "zon-norte",  
  "especialidad": "pavimento", "activa": true }
```

Por qué sincronizado existe. Un reporte se crea parado frente al pozo, donde puede no haber señal. Se guarda primero en el teléfono con `sincronizado: false` y se sube cuando vuelve la conexión. Esa cola local es parte del trabajo, no un detalle.

"El vecino no se enoja porque el bache tarde. Se enoja porque nadie le dice nada."

— Claudia, Centro de Atención al Vecino de Gualeguaychú